AI ASSI CODING-13.4

HTNO:2303A510A3

Batch:14

M. Sai Teja

Task 1: Refactoring Data Transformation

Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output. Legacy Code

```
values = [2, 4, 6, 8, 10]
doubled = [] for
v in values:
```

doubled.append(v * 2)

print(doubled) Expected

Output

[4, 8, 12, 16, 20]

Prompt:

#Legacy code for doubling values in a list

Code:

```
2.1.py > ...

#Legacy code for doubling values in a list
values = [2, 4, 6, 8, 10]

doubled = []
for v in values:
    doubled.append(v * 2)

print(doubled)
```

#Refactor the code using list comprehension

Code:

```python
#Refactor the code using list comprehension
values = [2, 4, 6, 8, 10]
doubled = [v * 2 for v in values]
print(doubled)
```

Output:

```
Drive/Documents/pthon learning/2.1.py"
[4, 8, 12, 16, 20]
[4, 8, 12, 16, 20]
PS C:\Users\nithe\OneDrive\Documents\pthon learning>
```

Justification:

1. Using a list comprehension simplifies the loop logic and produces the transformed list **in** a single readable line.

2. It follows Pythonic coding practices, making the code shorter, cleaner, and easier to maintain while keeping the same output.

Task 2: Improving Text Processing Code

Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping

the final output unchanged. Legacy Code words

= ["Refactoring", "with", "AI", "improves",

"quality"] message

= "" for w in words:

message += w + " "

print(message.strip

())

Expected Output

Refactoring with AI improves quality

Prompt:

#Legacy code for concatenating words into a message Code:

```python
#Legacy code for concatenating words into a message
words = ["Refactoring", "with", "AI", "improves", "quality"]

message = ""
for w in words:
    message += w + " "

print(message.strip())
```

#Refactor the code and more efficient and readable approach

```python
#Refactor the code and more efficient and readable  approach
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = " ".join(words)
print(message)
```

Output:

```
Refactoring with AI improves quality
Refactoring with AI improves quality
PS C:\Users\nithe\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

1. It is **more efficient than repeated string concatenation**, which creates many temporary strings inside the loop.
2. This approach follows **Pythonic coding practices**, making the code shorter, cleaner, and easier to maintain.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.

- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys. Legacy Code

config = {"host": "localhost", "port": 8080}

if "timeout" in config:

print(config["timeout"]) else:

print("Default timeout used")

Expected Output

Default timeout used

Prmpt:

#Legacy code for checking for a key in a dictionary values Code:

```
#Legacy code for checking for a key in a dictionary values
config = {"host": "localhost", "port": 8080}

if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

#Refactor the code using safer dictionary access methods and the behavior remains the same for missing keys.

```
#Refactor the code using  safer dictionary access methods and the behavior remains the same for missing keys.
config = {"host": "localhost", "port": 8080}
timeout = config.get("timeout", "Default timeout used")
print(timeout)
```

Output:

```
Default timeout used
Default timeout used
PS C:\Users\nithe\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

1. The **dict.get() method safely retrieves dictionary values** without raising an error if the key does not exist.

2. It **simplifies the code by removing manual key checking**, making the program shorter and more readable

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input. Task Description

- Examine the multiple if–elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality. Legacy Code action = "divide" x, y = 10, 2 if action == "add":

result = x + y elif action == "subtract":

result = x - y elif action == "multiply":

result = x * y elif action == "divide": result = x / y else:

result = None print(result)

Expected Output

5.0

Prompt:

#Legacy code multiple if-else conditions for operations Code:

```
#Legacy code multiple if-else conditions for operations
action = "divide"
x, y = 10, 2

if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None

print(result)
```

#Refactor the logic using mapping techniques while preserving functionality cleaner and scalable

```python
#Refactor the logic using mapping techniques while preserving functionality cleaner and scalable
action = "divide"
x, y = 10, 2

operations = {
    "add": x + y,
    "subtract": x - y,
    "multiply": x * y,
    "divide": x / y
}

result = operations.get(action, None)

print(result)
```

Justification:

1. Dictionary mapping **reduces multiple if–elif statements**, making the code shorter and more readable.
2. It **improves scalability**, because new operations can be added easily without changing the existing logic.

Task 5: Simplifying Search Logic in

Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.
- Use AI assistance to refactor the logic into a more concise and readable form.
- Maintain the same output behavior. Legacy

Code inventory = ["pen", "notebook", "eraser",

"marker"] found =

False for item in

inventory: if item ==

"eraser":

found = True break

print("Item Available" if found else "Item Not Available")

Expected Output Item

Available Legacy Code:

```python
#Legacy code
inventory = ["pen", "notebook", "eraser", "marker"]

found = False
for item in inventory:
    if item == "eraser":
        found = True
        break

print("Item Available" if found else "Item Not Available")
```

#refactor the logic into a more concise and readable form.

```python
#refactor the logic into a more concise and readable form.
inventory = ["pen", "notebook", "eraser", "marker"]

print("Item Available" if "eraser" in inventory else "Item Not Available")
```

Output:

```
Item Available
Item Available
PS C:\Users\nithe\AppData\Local\Programs\Microsoft VS Code>
```

Justification:

1. The **in operator directly checks if an element exists in the collection**, eliminating the need for a manual loop.

2. This **makes the code shorter, clearer, and easier to maintain while keeping the same functionality**.